Gold Layer Implementation

I implemented the Gold layer using a **star schema design** with fact and dimension tables built from the Silver layer.

Fact Tables

- **fact_sales**
  - Grain: One row per transaction line
  - Measures: sales_amount, quantity, profit, discount_amount
  - Purpose: Supports revenue analysis and inventory insights
- **fact_return**
  - Grain: One row per return event
  - Measures: refund_amount
  - Purpose: Supports return tracking and fraud detection

Dimension Tables

- **dim_customer** → customer attributes and segment
- **dim_product** → product details
- **dim_vendor** → vendor information
- **dim_region** → standardized regions
- **dim_date** → calendar attributes

These dimensions provide business context for analyzing facts.

Pre-Aggregated Tables (Key Requirement)

To improve performance and support frequent queries, I created:

- **agg_monthly_revenue_by_region**
  Monthly sales, profit, and order count
- **agg_customer_return_summary**
  Return count and refund amount per customer
- **agg_vendor_return_rate**
  Refund-to-sales ratio per vendor
- **agg_product_region_monthly**
  Product performance by region and month

Why Pre-Aggregation Was Used

These aggregations are:

- Frequently used in dashboards

- Based on large fact tables
- Expensive to compute repeatedly

Pre-computing them ensures:

- Faster query performance
- Reduced compute cost
- Consistent business metrics

Data Flow

- Bronze → Raw ingestion
- Silver → Cleaned and standardized data
- Gold → Analytical model (facts + dimensions + aggregations)

Challenges Faced

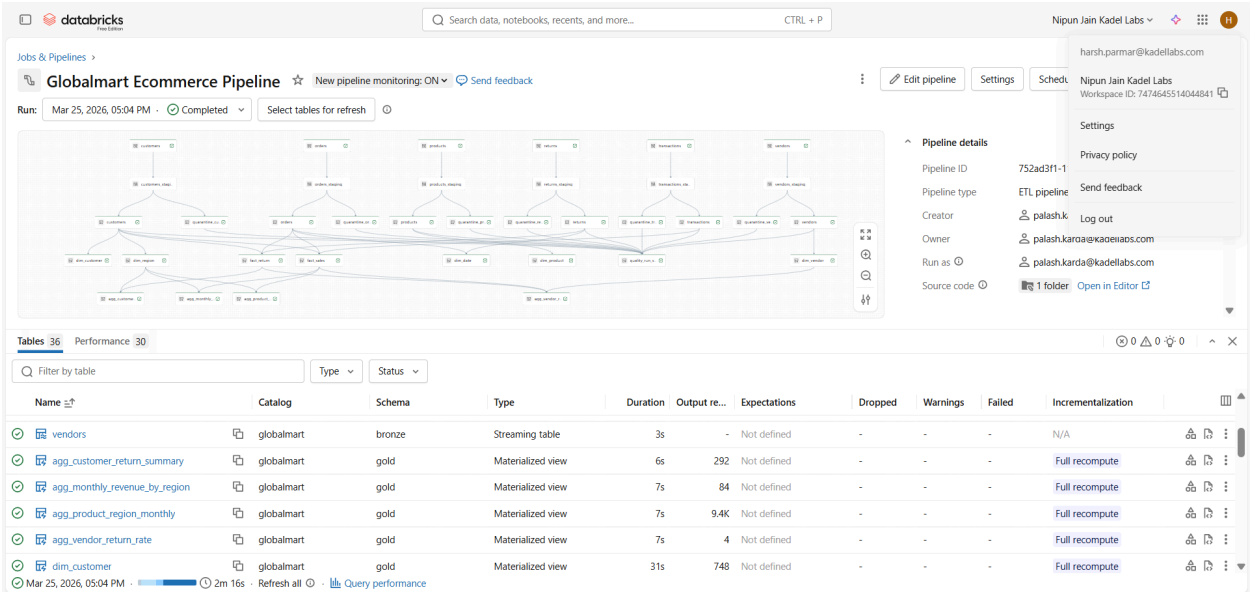
- **Join consistency:**
  Ensured correct joins using `order_id`, `customer_id`, and `vendor_id`
- **Schema mismatches:**
  Resolved differences by standardizing columns in Silver
- **Missing data:**
  Handled using NULL-safe logic and flags

Final Outcome

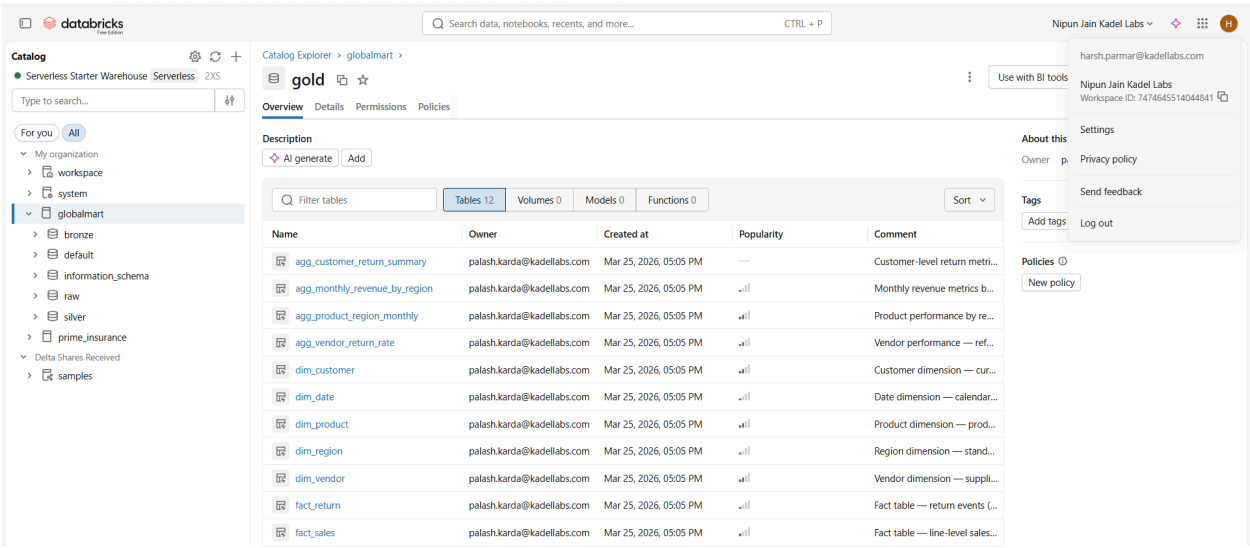
The Gold layer is:

- Fully populated with fact and dimension tables
- Optimized for analytics queries
- Ready for dashboards and GenAI use cases
- Automatically updated through the pipeline

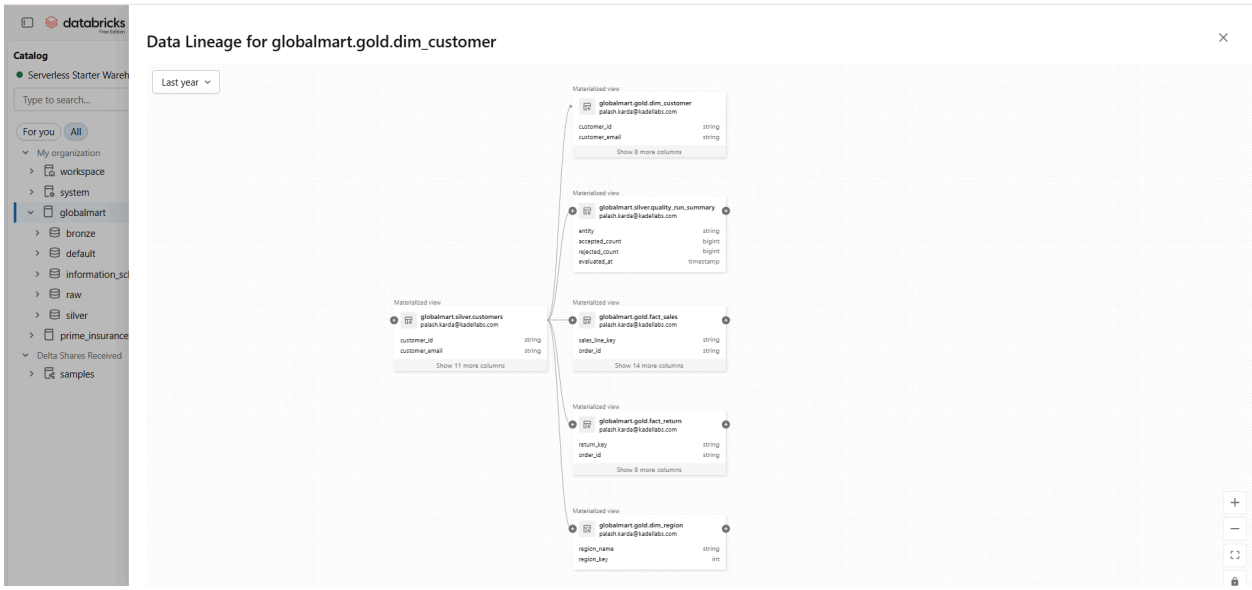
Gold Successful Pipeline Run



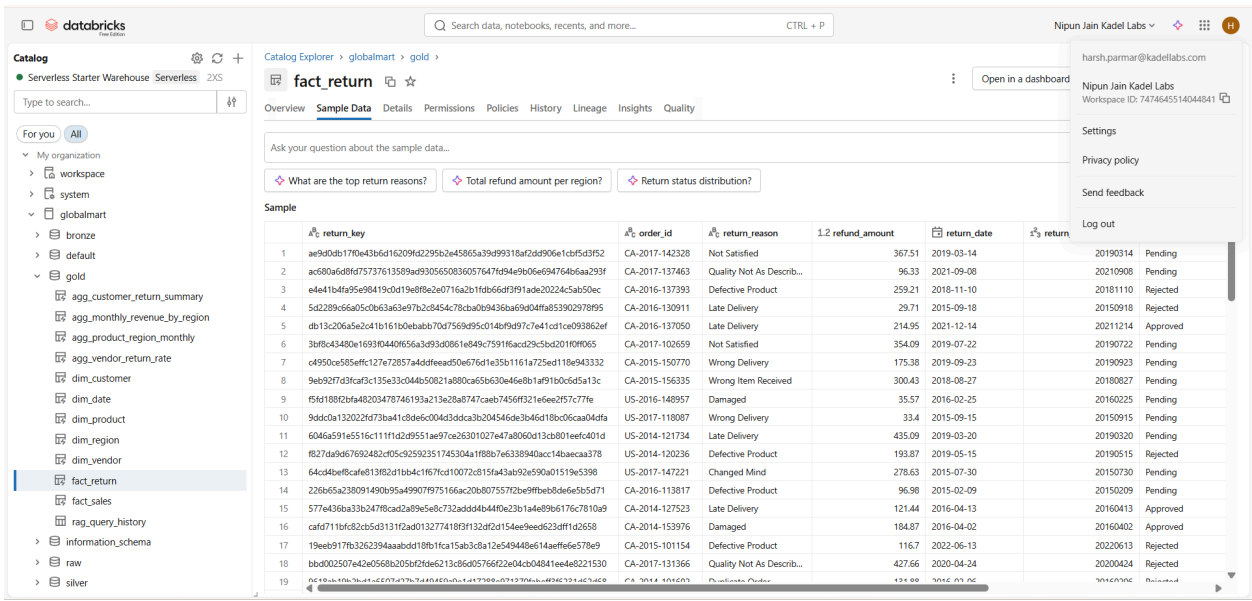
Gold Successful Pipeline Run



Gold Unity Catalog

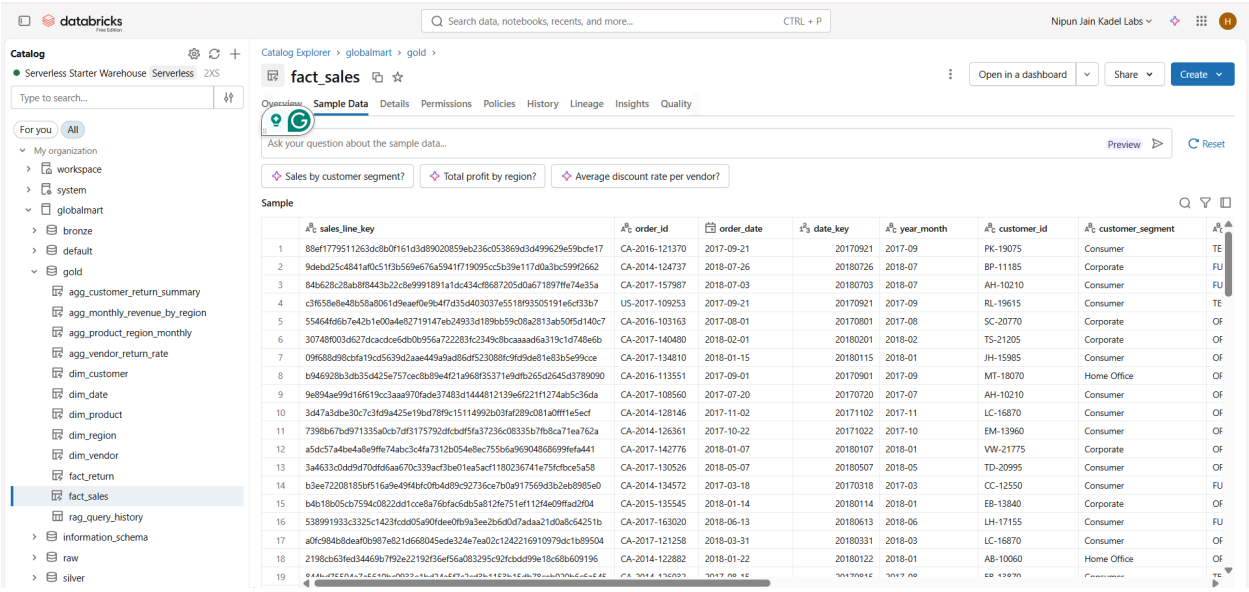


Customer Lineage

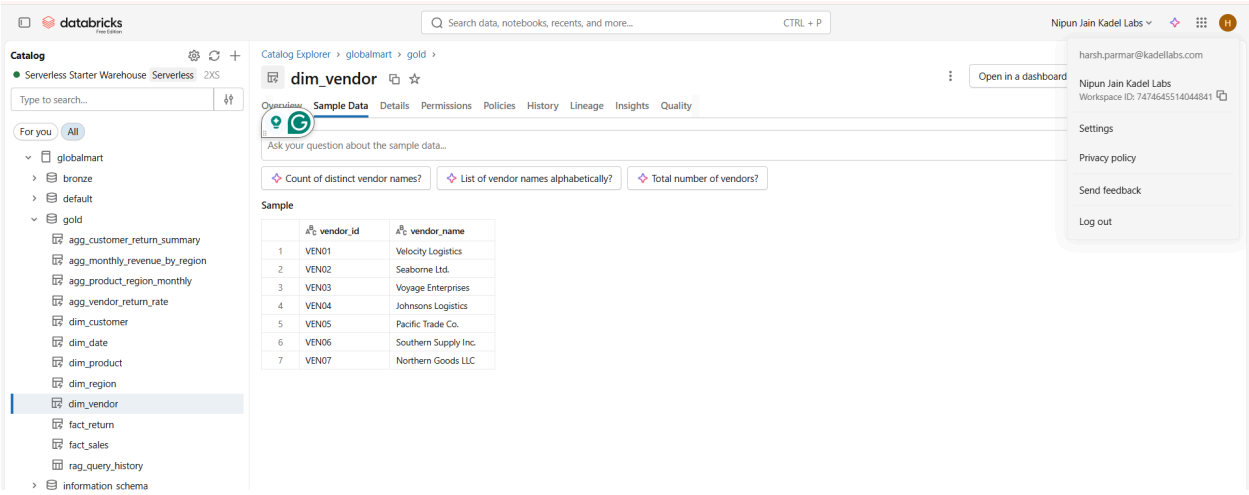


	return_key	order_id	return_reason	refund_amount	return_date		
1	ae9d0db17f0e43b6d16209fd2295b2e45865a39d99318af2dd906e1cbf5d3f52	CA-2017-142328	Not Satisfied	367.51	2019-03-14	20190314	Pending
2	ac680a6d8fd75737613589ad930565083605764 7fd94e9b06e694764b6aa293f	CA-2017-137463	Quality Not As Describ...	96.33	2021-09-08	20210908	Pending
3	e4e41b4fa95e98419c0d19e8f8e2e0716a2b1fdb66df3f91ade20224c5ab50ec	CA-2016-137393	Defective Product	259.21	2018-11-10	20181110	Rejected
4	5d2289c66a05c0b63a63e97b2c8454c78cba0b9436ba69d04ffa853902978f95	CA-2016-130911	Late Delivery	29.71	2015-09-18	20150918	Rejected
5	db13c206a5e2c41b161b0ebabb70d7569d95c014bf9d97c7e41cd1ce093862ef	CA-2016-137050	Late Delivery	214.95	2021-12-14	20211214	Approved
6	3bf8c43480e16931f0440f656a3d93d0861e849c7591f6acd29c5bd201f0ff065	CA-2017-102659	Not Satisfied	354.09	2019-07-22	20190722	Pending
7	c4950ce585effc127e72857a4ddfeead50e676d1e35b1161a725ed118e943332	CA-2015-150770	Wrong Delivery	175.38	2019-09-23	20190923	Pending
8	9eb92f7d3fcaf3c135e33c044b50821a880ca65b630e46e8b1af91b0c6d5a13c	CA-2015-156335	Wrong Item Received	300.43	2018-08-27	20180827	Pending
9	f5fd188f2bfa48203478746193a213e28a8747caeb7456ff321e6ee2f57c77fe	US-2016-148957	Damaged	35.57	2016-02-25	20160225	Pending
10	9ddc0a132022fd73ba41c8de6c004d3ddca3b204546de3b46d18bc06caa04dfa	US-2017-118087	Wrong Delivery	33.4	2015-09-15	20150915	Pending
11	6046a591e5516c111f1d2d9551ae97ce26301027e47a8060d13cb801eefc401d	US-2014-121734	Late Delivery	435.09	2019-03-20	20190320	Pending
12	f827da9d67692482cf05c92592351745304a1f88b7e6338940acc14baecaa378	US-2014-120236	Defective Product	193.87	2019-05-15	20190515	Rejected
13	64cd4bef8cafe813f82d1bb4c1f67fcd10072c815fa43ab92e590a01519e5398	US-2017-147221	Changed Mind	278.63	2015-07-30	20150730	Pending
14	226b65a238091490b95a49907f975166ac20b807557f2be9ffbeb8de6e5b5d71	CA-2016-113817	Defective Product	96.98	2015-02-09	20150209	Pending
15	577e436ba33b247f8cad2a89e5e8c732addd4b44f0e23b1a4e89b6176c7810a9	CA-2014-127523	Late Delivery	121.44	2016-04-13	20160413	Approved
16	cafd711bfc82cb5d3131f2ad013277418f3f132df2d154ee9eed623dff1d2658	CA-2014-153976	Damaged	184.87	2016-04-02	20160402	Approved
17	19eeb917fb3262394aaabdd18fb1fca15ab3c8a12e549448e614aeffe6e578e9	CA-2015-101154	Defective Product	116.7	2022-06-13	20220613	Rejected
18	bbd002507e42e0568b205bf2fde6213c86d05766f22e04cb04841ee4e8221530	CA-2017-131366	Quality Not As Describ...	427.66	2020-04-24	20200424	Rejected
19	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

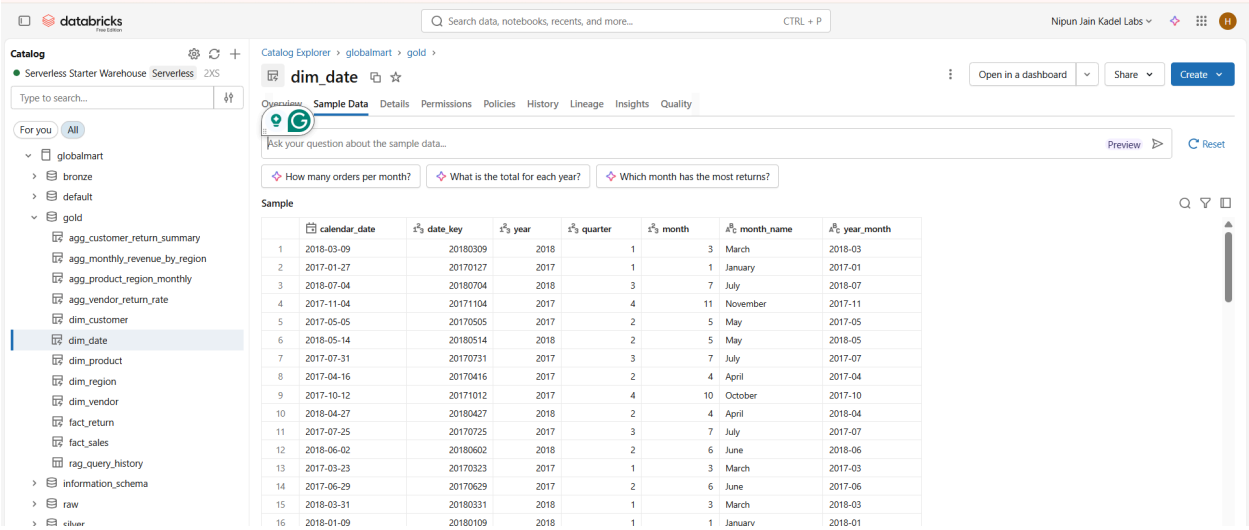
fact_return Data Display



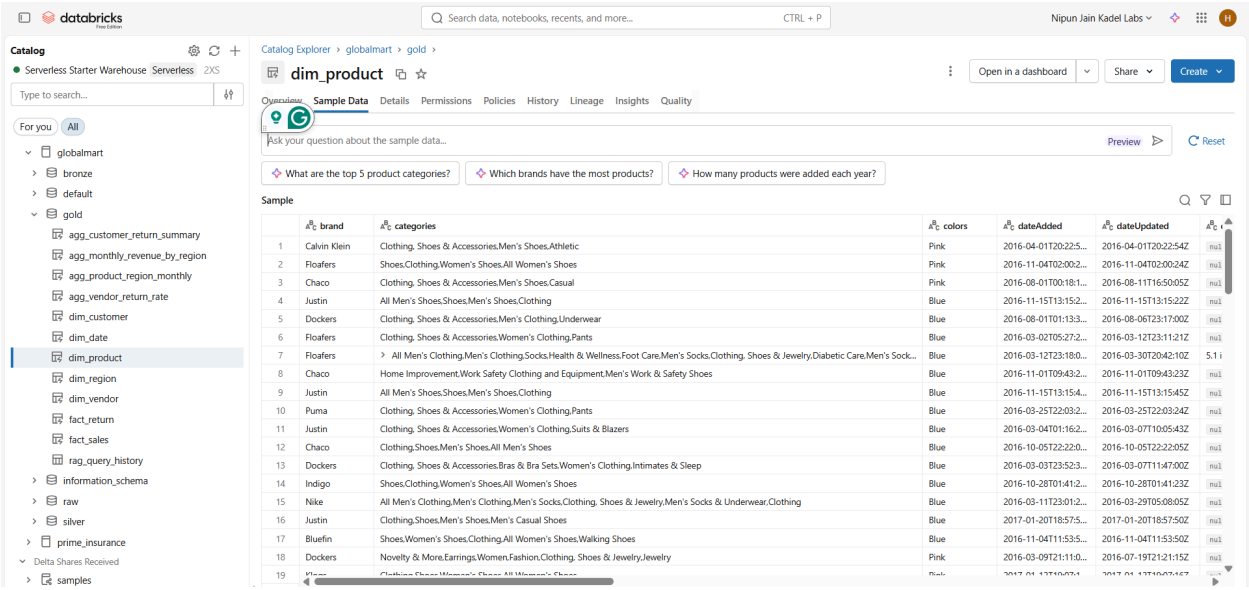
Fact_sales Data Display



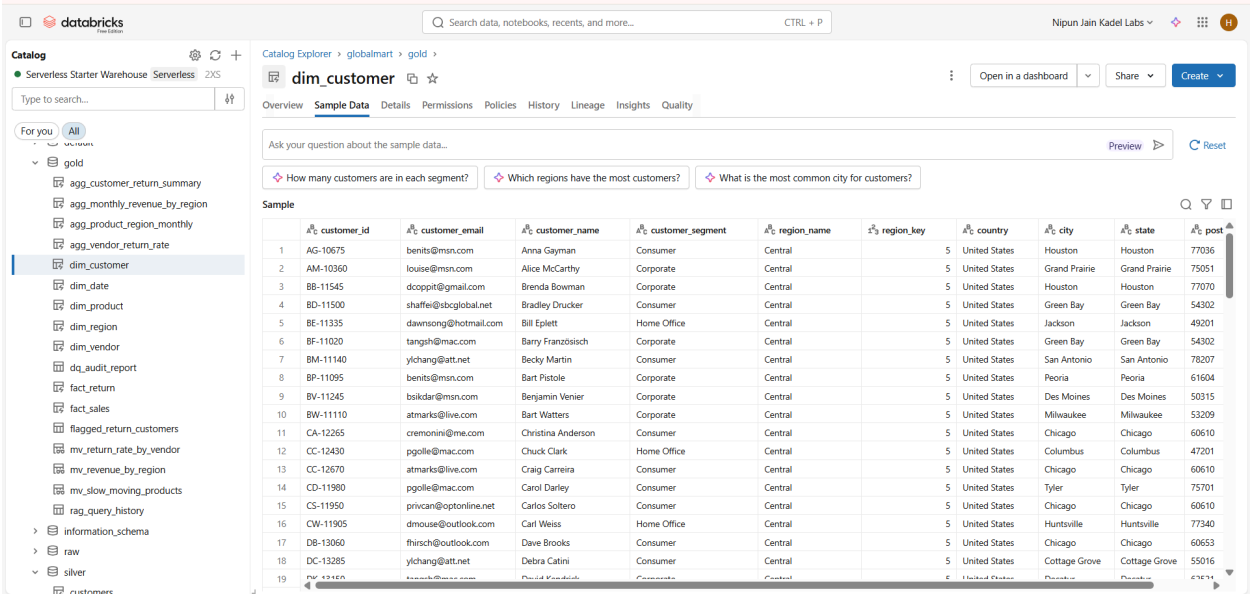
Dim_vendor data display



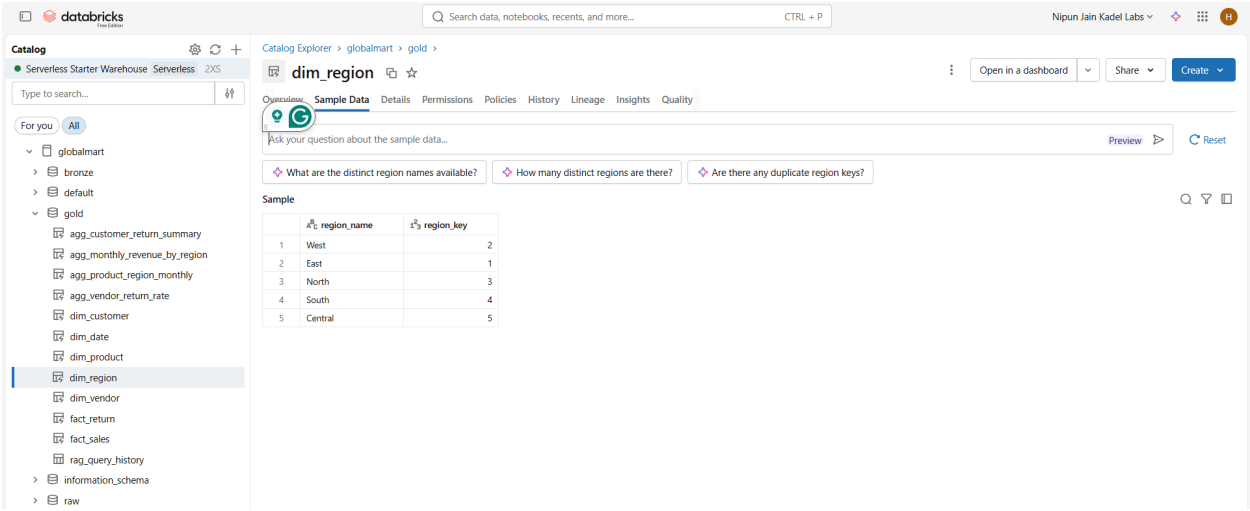
Dim_date data display



Dim_product data display



Dim_customer data display



Dim_region data display

Code -

```
# GLOBALMART — GOLD LAYER (Dimensional model + facts + aggregations)
```

```
# What this about
# - Dimensions: `dim_region`, `dim_date`, `dim_customer`, `dim_product`,
`dim_vendor`
# - Facts: `fact_sales`, `fact_return`
# - Pre-aggregations: `agg_monthly_revenue_by_region`,
`agg_customer_return_summary`, `agg_vendor_return_rate`,
`agg_product_region_monthly`

import dlt
from pyspark.sql import functions as F
from pyspark.sql.types import DoubleType

# Source (cleaned layer)
SILVER = "globalmart.silver"

# Target (analytical layer)
GOLD_CAT = "globalmart.gold"

# HELPER FUNCTION — REGION SURROGATE KEY
def region_key_expr(col):
    c = F.trim(col)
    return (
        F.when(c.isin("East"), F.lit(1))
        .when(c.isin("West"), F.lit(2))
        .when(c.isin("North"), F.lit(3))
        .when(c.isin("South"), F.lit(4))
        .when(c.isin("Central"), F.lit(5))
        .otherwise(F.lit(99))
    )

# DIMENSION: REGION
@dlt.table(
    name=f"{GOLD_CAT}.dim_region",
    comment="Region dimension — standardized list of regions for analysis"
)
def dim_region():
    """
```

```
    SOURCE:
    - globalmart.silver.customers

    OUTPUT:
    - region_key (PK)
    - region_name
    """

    c = spark.table(f"{SILVER}.customers")

    # Extract unique regions from customer data
    reg = (
        c.select(F.trim(F.col("region")).alias("region_name"))
        .distinct()
        .filter(F.col("region_name").isNotNull() & (F.col("region_name") !=
""))
    )

    # Add surrogate key for efficient joins
    return reg.withColumn("region_key", region_key_expr(F.col("region_name")))

# DIMENSION: DATE
@dlt.table(
    name=f"{GOLD_CAT}.dim_date",
    comment="Date dimension — calendar attributes for orders and returns"
)
def dim_date():
    """
    SOURCE:
    - globalmart.silver.orders
    - globalmart.silver.returns

    OUTPUT:
    - date_key (YYYYMMDD integer)
    - calendar_date
    - year, quarter, month
    - month_name
    - year_month (used in aggregations)
```

```
    """

    o = spark.table(f"{SILVER}.orders")
    r = spark.table(f"{SILVER}.returns")

    # Extract order dates
    od = o.select(
        F.to_date(F.col("order_purchase_date")).alias("calendar_date")
    ).filter(F.col("calendar_date").isNotNull())

    # Extract return dates
    rd = r.select(
        F.to_date(F.col("return_date")).alias("calendar_date")
    ).filter(F.col("calendar_date").isNotNull())

    # Combine all unique dates
    dates = od.unionByName(rd).distinct()

    # Generate calendar attributes
    return (
        dates.withColumn(
            "date_key",
            F.date_format(F.col("calendar_date"), "yyyyMMdd").cast("int")
        )
        .withColumn("year", F.year("calendar_date"))
        .withColumn("quarter", F.quarter("calendar_date"))
        .withColumn("month", F.month("calendar_date"))
        .withColumn("month_name", F.date_format("calendar_date", "MMMM"))
        .withColumn("year_month", F.date_format("calendar_date", "yyyy-MM"))
    )

# DIMENSION: CUSTOMER

@dlt.table(
    name=f"{GOLD_CAT}.dim_customer",
    comment="Customer dimension — current state (Type 1) with attributes for
segmentation and regional analysis"
)
```

```python
def dim_customer():
    """
    SOURCE:
    - globalmart.silver.customers

    OUTPUT:
    - customer_id (PK)
    - customer_email, customer_name
    - customer_segment
    - region_name, region_key
    - address attributes (country, city, state, postal_code)
    """

    c = spark.table(f"{SILVER}.customers")

    return c.select(
        F.col("customer_id"),
        F.col("customer_email"),
        F.col("customer_name"),

        # Standardized segment for analysis
        F.col("segment").alias("customer_segment"),

        # Region (string + surrogate key)
        F.trim(F.col("region")).alias("region_name"),
        region_key_expr(F.col("region")).alias("region_key"),

        # Additional attributes for filtering
        F.col("country"),
        F.col("city"),
        F.col("state"),
        F.col("postal_code"),
    )

# DIMENSION: PRODUCT
@dlt.table(
    name=f"{GOLD_CAT}.dim_product",
```

```
    comment="Product dimension — product catalogue for sales and inventory
analysis"
)
def dim_product():
    """
    SOURCE:
    - globalmart.silver.products

    OUTPUT:
    - product_id (PK)
    - product attributes (name, category, etc.)
    """

    p = spark.table(f"{SILVER}.products")

    # Remove pipeline/technical columns
    drop_cols = {"_source_file_path", "bronze_ingest_ts", "silver_load_ts"}

    cols = [x for x in p.columns if x not in drop_cols]

    return p.select(*cols) if cols else p

# DIMENSION: VENDOR
@dlt.table(
    name=f"{GOLD_CAT}.dim_vendor",
    comment="Vendor dimension — supplier reference for return rate and
performance analysis"
)
def dim_vendor():
    """
    SOURCE:
    - globalmart.silver.vendors

    OUTPUT:
    - vendor_id (PK)
    - vendor_name
    """
```

```
    v = spark.table(f"{SILVER}.vendors")

    return v.select(
        # Clean vendor_id and vendor_name
        F.trim(F.col("vendor_id")).alias("vendor_id"),
        F.trim(F.col("vendor_name")).alias("vendor_name"),
    )

# FACT TABLE: SALES

@dlt.table(
    name=f"{GOLD_CAT}.fact_sales",
    comment="Fact table — line-level sales (grain: one row per transaction
line)"
)
def fact_sales():
    """
    GRAIN:
    - One row per transaction line (order_id + product_id)

    SOURCE:
    - globalmart.silver.transactions (sales + quantity + profit)
    - globalmart.silver.orders (order date, vendor)
    - globalmart.silver.customers (segment, region)

    DIMENSION KEYS:
    - customer_id → dim_customer
    - product_id → dim_product
    - vendor_id → dim_vendor
    - region_key → dim_region
    - date_key → dim_date
    """

    t = spark.table(f"{SILVER}.transactions")
    o = spark.table(f"{SILVER}.orders")
    c = spark.table(f"{SILVER}.customers")

    # JOIN LOGIC
```

```
# Join transactions with orders to get order date and vendor_id
# Join with customers to get region and segment
j = (
    t.join(o, "order_id", "inner")
    .join(c, "customer_id", "inner")
    .select(
        F.col("order_id"),
        F.col("product_id"),
        F.col("sales_amount"),
        F.col("quantity"),
        F.col("discount"),
        F.col("profit"),
        F.col("profit_missing_flag"),
        F.col("vendor_id"),
        F.col("order_purchase_date"),
        F.col("customer_id"),

        # Customer attributes for analysis
        F.trim(F.col("region")).alias("customer_region"),
        F.col("segment").alias("customer_segment"),
    )
)

# DERIVED COLUMNS

j = j.withColumn(
    "order_date",
    F.to_date(F.col("order_purchase_date"))
).withColumn(
    # Used to join with dim_date
    "date_key",
    F.date_format(F.col("order_date"), "yyyyMMdd").cast("int"),
).withColumn(
    # Used for aggregations (monthly)
    "year_month",
    F.date_format(F.col("order_date"), "yyyy-MM"),
).withColumn(
    # Region surrogate key for joining dim_region
```

```
        "region_key",
        region_key_expr(F.col("customer_region")),
    ).withColumn(
        # Derived measure: discount amount
        "discount_amount",
        F.coalesce(F.col("sales_amount") * F.col("discount"), F.lit(0.0)),
    )

    # SURROGATE KEY (FACT PRIMARY KEY)
    # Unique identifier for each transaction line
    line_key = F.sha2(
        F.concat_ws(
            "|",
            F.col("order_id"),
            F.col("product_id"),
            F.col("sales_amount").cast("string"),
            F.col("quantity").cast("string"),
            F.coalesce(F.col("profit"), F.lit(0.0)).cast("string"),
        ),
        256,
    )

    # FINAL OUTPUT

    return j.select(
        line_key.alias("sales_line_key"),
        "order_id",
        "order_date",
        "date_key",
        "year_month",
        "customer_id",
        "customer_segment",
        "product_id",
        "vendor_id",
        "region_key",

        # Measures
        "sales_amount",
```

```
        F.col("quantity").cast(DoubleType()).alias("quantity"),
        F.col("profit").cast(DoubleType()).alias("profit"),
        "profit_missing_flag",
        F.col("discount").alias("discount_rate"),
        "discount_amount",
    )

# FACT TABLE: RETURNS

@dlt.table(
    name=f"{GOLD_CAT}.fact_return",
    comment="Fact table — return events (grain: one row per return)"
)
def fact_return():
    """
    GRAIN:
    - One row per return event

    SOURCE:
    - globalmart.silver.returns
    - globalmart.silver.orders
    - globalmart.silver.customers

    DIMENSION KEYS:
    - customer_id → dim_customer
    - vendor_id → dim_vendor
    - region_key → dim_region
    - return_date_key → dim_date
    """

    r = spark.table(f"{SILVER}.returns")
    o = spark.table(f"{SILVER}.orders")
    c = spark.table(f"{SILVER}.customers")

    # JOIN LOGIC
    # Join returns with orders to get vendor_id
    # Join with customers to get region
    j = (
```

```
        r.join(o, "order_id", "inner")
        .join(c, "customer_id", "inner")
        .select(
            F.col("order_id"),
            F.col("return_reason"),
            F.col("refund_amount"),
            F.col("return_date"),
            F.col("return_status"),
            F.col("vendor_id"),
            F.col("customer_id"),
            F.trim(F.col("region")).alias("customer_region"),
        )
    )

    # DERIVED COLUMNS

    j = j.withColumn(
        "return_dt",
        F.to_date(F.col("return_date"))
    ).withColumn(
        # Used to join with dim_date
        "return_date_key",
        F.date_format(F.col("return_dt"), "yyyyMMdd").cast("int"),
    ).withColumn(
        # Region surrogate key
        "region_key",
        region_key_expr(F.col("customer_region"))
    )

    # SURROGATE KEY (FACT PRIMARY KEY)

    rk = F.sha2(
        F.concat_ws(
            "|",
            F.col("order_id"),
            F.col("refund_amount").cast("string"),
            F.col("return_dt").cast("string"),
            F.col("return_status"),
```

```
        ),
        256,
    )

    # FINAL OUTPUT

    return j.select(
        rk.alias("return_key"),
        "order_id",
        "return_reason",
        F.col("refund_amount").cast(DoubleType()).alias("refund_amount"),
        F.col("return_dt").alias("return_date"),
        "return_date_key",
        "return_status",
        "vendor_id",
        "customer_id",
        "region_key",
    )

# GOLD LAYER — PRE-AGGREGATIONS

# AGGREGATION: MONTHLY REVENUE BY REGION

@dlt.table(
    name=f"{GOLD_CAT}.agg_monthly_revenue_by_region",
    comment="Monthly revenue metrics by region (sales, profit, order count)"
)
def agg_monthly_revenue_by_region():
    """
    BUSINESS PURPOSE:
    - Revenue audit and performance tracking
    - Used in dashboards to analyze monthly trends by region

    SOURCE:
    - fact_sales (transaction-level data)
    - dim_region (region names)

    GRAIN:
```

```
    - One row per (year_month, region)

    MEASURES:
    - total_sales
    - total_profit
    - order_count
    - total_quantity
    """

    f = dlt.read(f"{GOLD_CAT}.fact_sales")
    reg = dlt.read(f"{GOLD_CAT}.dim_region")

    # Aggregate sales metrics
    a = f.groupBy("year_month", "region_key").agg(
        F.sum("sales_amount").alias("total_sales"),
        F.sum("profit").alias("total_profit"),
        F.countDistinct("order_id").alias("order_count"),
        F.sum("quantity").alias("total_quantity"),
    )

    # Join with region dimension to get region name
    return a.join(reg, "region_key", "left")


# AGGREGATION: CUSTOMER RETURN SUMMARY

@dlt.table(
    name=f"{GOLD_CAT}.agg_customer_return_summary",
    comment="Customer-level return metrics for fraud detection"
)
def agg_customer_return_summary():
    """
    BUSINESS PURPOSE:
    - Identify customers with high return activity
    - Support fraud detection and policy decisions

    SOURCE:
    - fact_return
```

```
    - dim_region

    GRAIN:
    - One row per (customer_id, region)

    MEASURES:
    - return_count
    - total_refund_amount
    - avg_refund_amount
    """

    fr = dlt.read(f"{GOLD_CAT}.fact_return")
    reg = dlt.read(f"{GOLD_CAT}.dim_region")

    # Aggregate return metrics
    a = fr.groupBy("customer_id", "region_key").agg(
        F.count("*").alias("return_count"),
        F.sum("refund_amount").alias("total_refund_amount"),
        F.avg("refund_amount").alias("avg_refund_amount"),
    )

    # Join with region dimension
    return a.join(reg, "region_key", "left")


# AGGREGATION: VENDOR RETURN RATE

@dlt.table(
    name=f"{GOLD_CAT}.agg_vendor_return_rate",
    comment="Vendor performance — refund to sales ratio"
)
def agg_vendor_return_rate():
    """
    BUSINESS PURPOSE:
    - Evaluate vendor performance
    - Identify vendors with high return rates

    SOURCE:
```

```
- fact_sales (sales data)
- fact_return (refund data)
- dim_vendor

GRAIN:
- One row per vendor

MEASURES:
- vendor_gross_sales
- vendor_refunds
- refund_to_sales_ratio
"""

fs = dlt.read(f"{GOLD_CAT}.fact_sales")
fr = dlt.read(f"{GOLD_CAT}.fact_return")
v = dlt.read(f"{GOLD_CAT}.dim_vendor")

# Total sales per vendor
sales = fs.groupBy("vendor_id").agg(
    F.sum("sales_amount").alias("vendor_gross_sales")
)

# Total refunds per vendor
refunds = fr.groupBy("vendor_id").agg(
    F.sum("refund_amount").alias("vendor_refunds")
)

# Combine sales and refunds
out = sales.join(refunds, "vendor_id", "left").fillna(
    {"vendor_refunds": 0.0}
)

# Calculate return rate (important business metric)
out = out.withColumn(
    "refund_to_sales_ratio",
    F.when(
        F.col("vendor_gross_sales") > 0,
        F.col("vendor_refunds") / F.col("vendor_gross_sales")
```

```
        ).otherwise(F.lit(None))
    )

    # Join with vendor dimension for context
    return out.join(v, "vendor_id", "left")


# AGGREGATION: PRODUCT PERFORMANCE

@dlt.table(
    name=f"{GOLD_CAT}.agg_product_region_monthly",
    comment="Product performance by region and month"
)
def agg_product_region_monthly():
    """
    BUSINESS PURPOSE:
    - Identify slow-moving and high-performing products
    - Support inventory planning and optimization

    SOURCE:
    - fact_sales
    - dim_region

    GRAIN:
    - One row per (product_id, region, month)

    MEASURES:
    - total_sales
    - total_quantity
    - total_profit
    - order_count
    """

    f = dlt.read(f"{GOLD_CAT}.fact_sales")
    reg = dlt.read(f"{GOLD_CAT}.dim_region")

    # Aggregate product-level metrics
    a = f.groupBy("year_month", "region_key", "product_id").agg(
```

```
        F.sum("sales_amount").alias("total_sales"),
        F.sum("quantity").alias("total_quantity"),
        F.sum("profit").alias("total_profit"),
        F.countDistinct("order_id").alias("order_count"),
    )

    # Join with region dimension
    return a.join(reg, "region_key", "left")
```